arXiv:2407.02209v1 [cs.CL] 2 Jul 2024

Generative Monoculture in Large Language Models

Fan Wu[1], **Emily Black**[2], **Varun Chandrasekaran**[1]
[1] University of Illinois Urbana-Champaign [2] Barnard College

Abstract

We introduce *generative monoculture*, a behavior observed in large language models (LLMs) characterized by a significant narrowing of model output diversity relative to available training data for a given task: for example, generating only positive book reviews for books with a mixed reception. While in some cases, generative monoculture enhances performance (e.g., LLMs more often produce efficient code), the dangers are exacerbated in others (e.g., LLMs refuse to share diverse opinions). As LLMs are increasingly used in high-impact settings such as education and web search, careful maintenance of LLM output diversity is essential to ensure a variety of facts and perspectives are preserved over time. We experimentally demonstrate the prevalence of generative monoculture through analysis of book review and code generation tasks, and find that simple countermeasures such as altering sampling or prompting strategies are insufficient to mitigate the behavior. Moreover, our results suggest that the root causes of generative monoculture are likely embedded within the LLM's alignment processes, suggesting a need for developing fine-tuning paradigms that preserve or promote diversity.

1 Introduction

Large language models (LLMs) show promise due to their emergent abilities [1] and state-of-the-art performance on several NLP tasks [2]. However, concerns have been raised about the increasing reliance on LLM-based systems with insufficient testing, and how they impact society [3, 4]. Recent evidence has shown that LLMs have dangerous tendencies: they convincingly return incorrect information [5, 6, 7], produce toxic language [8, 9], and can effectively propagate misinformation [10, 11].

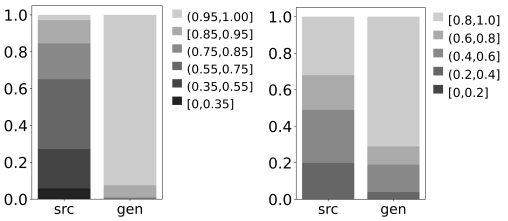

Figure 1: **(Left)** Comparison of the range of average-per-book sentiment scores for book reviews generated by an LLM (`gen`) and by human reviewers from the Goodreads dataset (`src`). *Note the generated reviews have a much smaller range, as they are overwhelmingly positive*. Model: `Llama-2-chat`. **(Right)** The spectrum of the mean pairwise Jaccard similarity among the algorithms of coding solutions. Note the generated code covers a narrower range of algorithms. Model: `GPT-4`

In this paper, we focus on a different concern: that for a given prompt and task, *LLMs do not faithfully represent the diversity of potential responses available in their training data*. We call this behavior *generative monoculture*: given some task (e.g., generating book reviews) and a data attribute (e.g. sentiment), generative monoculture refers to the narrowing of the probability distribution of the considered attribute from source data (i.e., available human-generated book reviews as part of the training data) to the generated data (i.e., LLM-generated book reviews).

As a preview, for book reviews (Fig. 1(**Left**)), we compare the diversity in sentiment of Goodreads reviews [12] (i.e., `src`)—very likely a portion of LLM training data [13]—with LLM-generated reviews (i.e., `gen`). The range of mean sentiment scores per book across `gen` book reviews is much narrower than that in the `src`: in the experiment pictured, the average sentiment score for `gen`

Preprint. Under review.

reviews are mostly over 0.85, whereas the the average sentiment score for `src` reviews over the same books have a wider range from zero to one. For the code generation task (Fig. 1(**Right**)), the range of algorithms employed in (correct) solutions to a given coding problem (i.e., `gen`) was much less varied than a sample of human answers (`src`) available on the web [14]: we show this through the range of Jaccard similarity of the algorithms employed in sets of human-generated (`src`) as well as LLM-generated (`gen`) responses to a coding prompt.

Through the rapid adoption of LLMs such as ChatGPT, CoPilot, and Devin across education, code generation, and day-to-day information gathering, generative monoculture can harm society through loss of information, creativity, and intellectual diversity. For example, students asking LLMs questions about class material to get help researching for an essay may have their opinions formed without exposure to a sufficiently wide subset of available information; this will allow for certain opinions to die out over time. Concretely, a reduction in diversity of sentiment displayed above may lead to the loss of arguments from negative opinions on controversial books, potentially crucial for historical or literary context or a nuanced understanding of a book's contributions.

Generative monoculture could even lead to security threats, depending on the application: software engineers across the globe relying on ChatGPT and CoPilot receiving similar code generations which do not reflect the true diversity of methods to solve a given problem may lead to similar code vulnerabilities across several large tech companies. Indeed, as we preview in Fig. 1 and show in detail in § 6, LLM output exhibits are less diverse than human-generated solutions in the training data (e.g., by employing a narrower array of algorithms), which could lead to similarity across a wide range of code bases, leading in turn to repeated vulnerabilities [15, 16]. However, in this case, LLM outputs not reflecting the full diversity of available coding examples can be positive: LLM outputs over-represent correct and efficient solutions. We present a nuanced picture of generative monoculture: while it can highlight the optimal portion of human-generated data for some attributes, its pervasiveness across generation tasks and attributes *may* cause harm without careful intervention.

In this paper, we (1) define the concept of generative monoculture, and compare it to prior work around related topics (§ 2 and 3); (2) introduce a paradigm for measuring generative monoculture in LLMs (§ 4); (3) show experimental evidence for the prevalence of generative monoculture across a variety of application areas (book reviews and code) and (open-source and proprietary) LLMs, and provide some evidence for what may exacerbate generative monoculture, such as alignment tuning [17], (§ 6 and 7); and (4) show the (in)efficacy of several methods to abate generative monoculture; namely, changing temperature, sampling, and prompting techniques (§ 6 and 7).

2 Defining Generative Monoculture

We broadly characterize generative monoculture as a *distribution shift* from source data (i.e., human-generated training data) to model generated data (i.e., model outputs) for a specific task, such as generating reviews for books or solutions to coding problems. This can be formalized using measures of statistical dispersion applied to various task-specific attributes.

Definition 1 (Generative Monoculture)**.** For a given task, let $\mathcal{P}_{\texttt{src}}$ denote the probability distribution of the source data, $\mathcal{P}_{\texttt{gen}}$ denote the probability distribution of the LLM-generated data, h denote a function extracting attributes from data (such as sentiment, or algorithms used in code), and Dispersion$(\cdot)$ denote a dispersion metric (e.g., entropy). Then we define generative monoculture as the condition where $\mathcal{P}_{\texttt{gen}}$ is statistically narrower than $\mathcal{P}_{\texttt{src}}$, namely: Dispersion$(h(x)|x \sim \mathcal{P}_{\texttt{gen}}) <$ Dispersion$(h(x)|x \sim \mathcal{P}_{\texttt{src}})$.

Note, $\mathcal{P}_{\texttt{src}}$/ $\mathcal{P}_{\texttt{gen}}$ can be the distribution of human-generated/model-generated responses, for a given task, conditioned on one specific given prompt (which we refer to as the *conditional distribution*), or the distribution of human-generated/model-generated responses for *any possible prompt* in a considered domain (which we call the *unconditional distribution*).

This phenomenon signifies a shift towards less varied outputs. We emphasize that the investigation of generative monoculture is intrinsically task-dependent, as the attributes of interest differ across tasks. In addition, as we often do not have access to the source distribution in practice, we approximate it using a source dataset ($D_{\texttt{src}}$), comprised of a subset of the training data of the LLMs we consider. Similarly, we approximate the generated distribution through a dataset generated by the model ($D_{\texttt{gen}}$).

Generative Monoculture, Human Preference, and Alignment: Generative monoculture can cause LLMs to over-emphasize human-preferred areas of a distribution for a certain data attribute; this is often desired behavior. For example, as we demonstrate in § 7, generative monoculture can result in having a narrower distribution of code correctness or efficiency biased towards correct, fast, and low-memory code. We conjecture this is a consequence of alignment procedures such as reinforcement learning with human feedback (RLHF) [17].

However, when a tendency stemming from human preference bleeds beyond its intended use—e.g., a preference for positive sentiment affecting outputs that need not or should not be positive—these seemingly advantageous behaviors can prevent an equally important goal: maintaining diversity of human opinion and expression. Further, along data attributes which do not have a clear preferred area of the distribution, generative monoculture can limit the scope of methods, topics, or ideas expressed.

3 Related Work

Diversity and LLMs. Santurkar *et al.* [18] demonstrate that LLMs do not give representative opinions to polling questions when compared to the general U.S. population. Our work focuses on the narrowing of diversity in LLM output from its human-generated training data—while Santukar *et al.* demonstrate a narrowing in diversity from actual human survey respondents (and not training data). Additionally, our work proposes a general framework for measuring monoculture. Padmakumar and He [19] demonstrate that using LLM assistance can lead to reduced diversity in human-generated argumentative essays when compared to essays written without LLM assistance. While they mention that this is partially because the models themselves do not produce diverse output, they do not focus on the narrowing of diversity from LLM training data to LLM-generated data. Finally, Sorensen *et al.* [20] outline an alignment framework to emphasize *pluralism*, to work towards creating models which express a variety of opinions and perspectives. While this is certainly related to our work, generative monoculture as a phenomenon extends beyond differences in opinion, and expresses the narrowing of any number of task-specific attributes, from code correctness to topics covered to many others. One common thread across many of these works, which our work adds to, is that *current alignment practices—namely RLHF— harms output diversity*.

Other Notions of Monoculture. Our notion of generative monoculture relates to, but differs from, other notions of monoculture in the AI literature. For example, algorithmic monoculture [21] and outcome homogeneity [22] describe the societal state where many decision-making actors rely on the same underlying algorithms to generate (classification or ranking) predictions, from the perspective of decision-making actors and individuals subject to those decisions respectively. These works show that algorithmic monoculture is sub-optimal for both decision-making actors (due to correlated failures across models) and for those subject to model decisions, as repeated outcomes across models leave little room for algorithmic recourse. In contrast, generative monoculture focuses on documenting the phenomenon of individual LLMs narrowing the diversity of their output in relation to their source data—for example, only returning positive book reviews about a controversial book. We do, however, document in this work that generative monoculture exists to similar extents and in similar directions across a variety of available LLMs, (e.g., Llama, Vicuna, ChatGPT-4) leaving open the possibility of concerns brought up by Kleinberg and Raghavan [21], but in a generative context.

4 Measuring Generative Monoculture

We outline a general approach to measuring generative monoculture in LLMs. In particular, following Definition 1, we outline steps to construct $D_{\texttt{src}}$ and $D_{\texttt{gen}}$ and compare their diversity through extracting data attributes and calculating dispersion metrics. We illustrate our approach in Fig. 2.

4.1 Data Curation

For a given task, we aim to create a source dataset that is likely to have been used in training the LLM we wish to investigate. Training data for most LLMs is a closely guarded secret. While recent work [23] describes how dataset contamination can be determined, such approaches are (a) riddled with false positives, and (b) computationally expensive. Thus, we often take an educated guess